Machine Learning in Python

Lecture 1, part 2: The perceptron

Przemysław Rola

Department of Mathematics, Krakow University of Economics

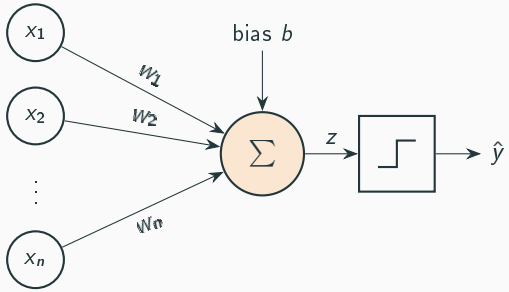
Plan

1. The model
2. Learning rule
3. Convergence and its limits
4. An alternative: support vector machines
5. Towards multilayer networks
6. In Python

The model

From a biological neuron to a threshold unit

- A neuron receives signals from many others, weighs them, and "fires" if the total stimulation exceeds a threshold.
- McCulloch & Pitts (1943) turned this into a logical unit; Rosenblatt (1958) made the weights learnable.
- The analogy is loose: modern networks are mathematical models, not brain simulations.



Definition

Inputs $x_1, \ldots, x_n$ (numbers), weights $w_1, \ldots, w_n$, bias b.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \qquad \text{(weighted sum)}$$

$$\hat{y} = \begin{cases} 1, & \text{if } z \geq 0, \\ 0, & \text{if } z < 0 \end{cases} \qquad \text{(step function } H(z)\text{)}$$

Definition

Inputs $x_1, \ldots, x_n$ (numbers), weights $w_1, \ldots, w_n$, bias b.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \qquad \text{(weighted sum)}$$

$$\hat{y} = \begin{cases} 1, & \text{if } z \geq 0, \\ 0, & \text{if } z < 0 \end{cases} \qquad \text{(step function } H(z)\text{)}$$

Toy example: credit decision (weights invented)

x_1 = monthly income, x_2 = monthly debt payments (thousand PLN). $w_1 = 1$, $w_2 = -2$, $b = -3$:

$$z = x_1 - 2x_2 - 3.$$

Client A: $x_1 = 8$, $x_2 = 2$: $z = 8 - 4 - 3 = 1 \geq 0 \Rightarrow \hat{y} = 1$ (approve).
Client B: $x_1 = 6$, $x_2 = 2$: $z = 6 - 4 - 3 = -1 < 0 \Rightarrow \hat{y} = 0$ (reject).

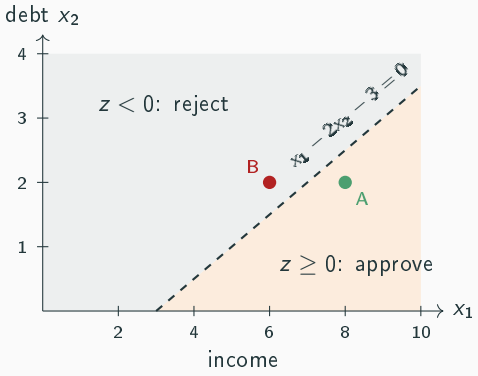
Machine learning: find w_1, w_2, b from past data instead of inventing them.

Geometry: a perceptron draws a straight line

- With two inputs the points where $z = 0$,

$$w_1 x_1 + w_2 x_2 + b = 0,$$

  form a straight line — the **decision boundary**.
- On one side $z > 0$ (answer 1), on the other $z < 0$ (answer 0).
- Solving for x_2 (if $w_2 \neq 0$):
  $x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$.
- With more inputs: a plane / hyperplane. The perceptron is a **linear classifier**.



Example: logic gates with hand-chosen weights

Inputs $x_1, x_2 \in \{0, 1\}$.

Gate	w_1	w_2	b
AND	1	1	-1.5
OR	1	1	-0.5
NAND	-1	-1	1.5

Check AND: $z(1,1) = 0.5 \geq 0$,
$z(1,0) = z(0,1) = -0.5 < 0$,
$z(0,0) = -1.5 < 0$.

- Many choices of weights work — any line separating the two groups.
- **Question**: can we find the weights *automatically* from examples?
- **Question**: is there a gate that *no* choice of (w_1, w_2, b) implements?

Learning rule

The perceptron learning algorithm (Rosenblatt)

Training data: examples with inputs $x_1, \ldots, x_n$ and the correct answer $d \in \{0, 1\}$. Learning rate $\eta > 0$ (e.g. $\eta = 1$).

1. Start with all weights and the bias equal to 0.
2. Go through the examples one by one (one pass = one **epoch**). For each example:
   - compute the answer $\hat{y}$ of the perceptron;
   - compute the error $e = d - \hat{y}$ $\quad (e \in \{-1, 0, 1\})$;
   - update every weight and the bias:

$$w_i \leftarrow w_i + \eta \cdot e \cdot x_i, \qquad b \leftarrow b + \eta \cdot e.$$

3. Stop when a whole epoch has no mistakes, or after a fixed number of epochs.

If the answer is correct, $e = 0$ and nothing changes. The perceptron learns only from its mistakes.

Why does the update make sense?

correct d	answer $\hat{y}$	error e	what the update does
0	0	0	nothing
1	1	0	nothing
1	0	1	adds ηx_i to w_i and η to $b \Rightarrow z$ goes up
0	1	-1	subtracts ηx_i from w_i and η from $b \Rightarrow z$ goes down

Case $d = 1$, $\hat{y} = 0$ with two inputs: the new value of z for the same example is

$$(w_1 + \eta x_1)x_1 + (w_2 + \eta x_2)x_2 + (b + \eta) = z + \eta\,(x_1^2 + x_2^2 + 1) > z.$$

So the perceptron moves *towards* the correct answer for this example.

Caveat: one step may not be enough, and it may spoil other examples. Does the process ever stop? → next section.

Worked example: learning OR ($\eta = 1$, start from $w_1 = w_2 = b = 0$)

epoch	(x_1, x_2)	d	(w_1, w_2), b before	z	$\hat{y}$	e	(w_1, w_2), b after
1	$(0,0)$	0	$(0,0)$, 0	0	1	-1	$(0,0)$, -1
1	$(0,1)$	1	$(0,0)$, -1	-1	0	$+1$	$(0,1)$, 0
1	$(1,0)$	1	$(0,1)$, 0	0	1	0	—
1	$(1,1)$	1	$(0,1)$, 0	1	1	0	—
2	$(0,0)$	0	$(0,1)$, 0	0	1	-1	$(0,1)$, -1
2	$(0,1)$	1	$(0,1)$, -1	0	1	0	—
2	$(1,0)$	1	$(0,1)$, -1	-1	0	$+1$	$(1,1)$, 0
2	$(1,1)$	1	$(1,1)$, 0	2	1	0	—

Epoch 3: one more mistake on $(0,0)$ gives $w_1 = w_2 = 1$, $b = -1$. Epoch 4: no mistakes $\Rightarrow$ stop.

Note: the final line $x_1 + x_2 = 1$ passes *through* $(0,1)$ and $(1,0)$ (they get $z = 0$, hence $\hat{y} = 1$). The perceptron stops at *some* separating line, not necessarily a "good" one — it does not look for the "safest" line (we come back to this with SVM).

Convergence and its limits

Does the perceptron always stop?

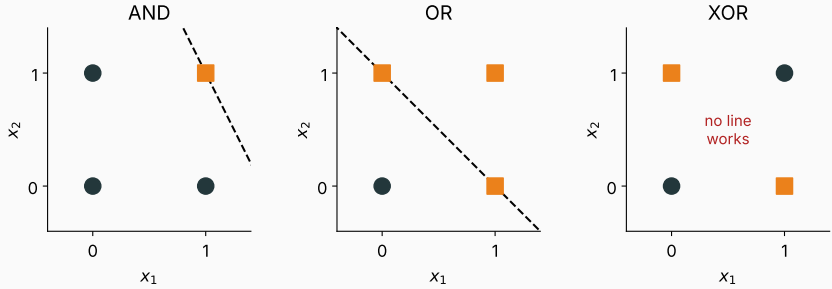
Convergence theorem (Rosenblatt; Block 1962; Novikoff 1962) — informal version

If there *exists* a straight line (hyperplane) that separates the two classes without errors, the perceptron algorithm finds such a line after a finite number of updates, whatever the order of the examples.

- Such data are called **linearly separable**.
- The more "room" there is between the classes, the smaller the guaranteed maximum number of updates.
- The theorem says nothing about *which* separating line we get.
- Precise statement and a short proof: appendix (optional).

What if the data are not linearly separable?

- The theorem does not apply: the algorithm never stops making mistakes and the weights keep changing. In practice: cap the number of epochs (`max_iter`), or use the *pocket* / *averaged* perceptron.
- XOR is the smallest example: no line separates $\{(0,1),(1,0)\}$ from $\{(0,0),(1,1)\}$.



AND and OR: boundaries found by the perceptron rule. XOR: no separating line exists.

XOR is not linearly separable — a two-line proof

Suppose $H(w_1 x_1 + w_2 x_2 + b)$ computes XOR. Then

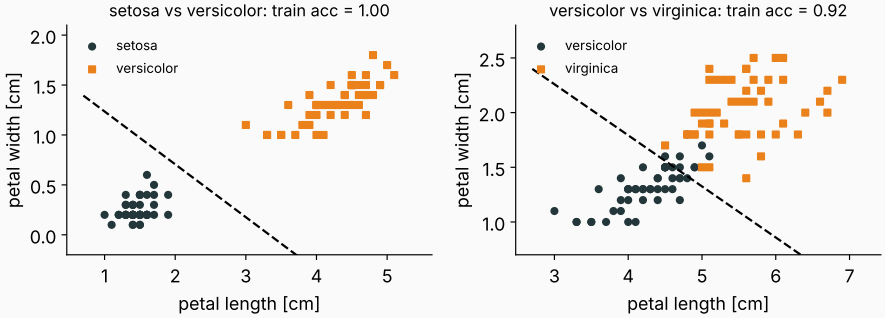
$$(0,0) \mapsto 0: \quad b < 0, \qquad (1,1) \mapsto 0: \quad w_1 + w_2 + b < 0,$$
$$(1,0) \mapsto 1: \quad w_1 + b \geq 0, \qquad (0,1) \mapsto 1: \quad w_2 + b \geq 0.$$

Adding the two inequalities in the first row: $w_1 + w_2 + 2b < 0$.
Adding the two inequalities in the second row: $w_1 + w_2 + 2b \geq 0$. Contradiction. $\square$

Historical note

Minsky & Papert (1969) analysed such limitations of single-layer perceptrons systematically (e.g. parity, connectedness). That several layers suffice was known, but there was no practical way to *train* them until backpropagation became widespread in the 1980s.

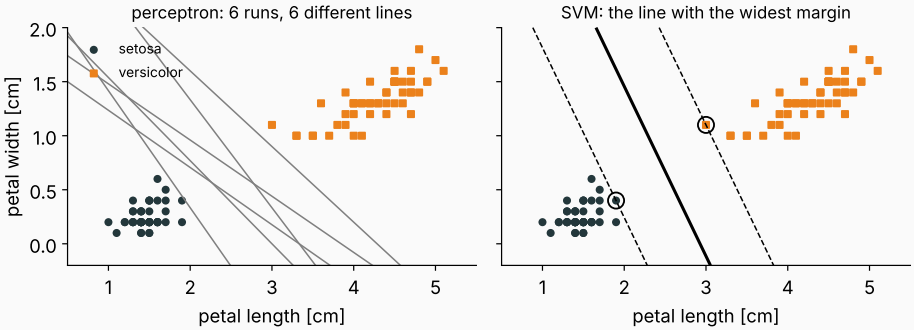
Real data: Iris (petal length and width)



Left: separable classes, the perceptron reaches 100% training accuracy. Right: the classes overlap, no line is perfect, and the line found depends on the order of the data and on when training is stopped.

An alternative: support vector machines

Which separating line is the best?

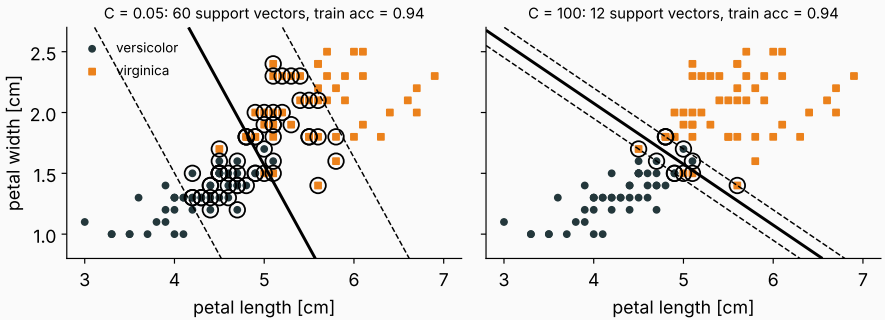


The perceptron stops at the *first* line without mistakes, so different runs give different lines. A **support vector machine** (SVM; Cortes & Vapnik 1995) chooses the line that is as far as possible from the nearest points of both classes.

SVM: the idea

- **Margin**: the empty band around the line (dashed lines). SVM chooses the line with the widest margin.
- **Support vectors**: the points lying on the edge of the margin (circled). The line depends only on them — moving other points (without crossing the margin) does not change it.
- Intuition: a wide margin leaves room for new observations that are slightly different from the training data.
- Training solves a convex optimisation problem (no local traps): the result does *not* depend on the order of the data or on a random seed.

Overlapping classes: soft margin and the parameter C



When no line is perfect, SVM allows some points inside the margin or on the wrong side, with a penalty controlled by C: **small** C — wide margin, many violations tolerated; **large** C — narrow margin, violations are expensive. C is a **hyperparameter**: we choose it, e.g. by checking accuracy on validation data.

SVM in scikit-learn

```
from sklearn.svm import SVC

svm = SVC(kernel="linear", C=1.0)
svm.fit(X_train, y_train)
print(svm.score(X_test, y_test))        # accuracy on new data
print(svm.support_vectors_)             # the circled points
```

- SVM is sensitive to the scale of the features ⇒ usually combined with `StandardScaler`.
- `kernel="rbf"` gives curved boundaries ("kernel trick"); not needed for Iris, we skip the details.

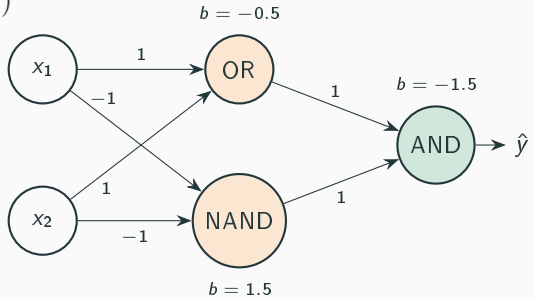
	perceptron	linear SVM
separable data	some separating line	line with the widest margin
overlapping data	does not settle, depends on order	best compromise for a given C

Towards multilayer networks

XOR with two layers

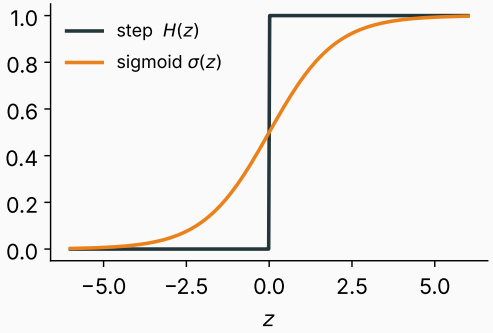
$$\mathrm{XOR}(x_1, x_2) = \mathrm{AND}\big(\mathrm{OR}(x_1, x_2),\ \mathrm{NAND}(x_1, x_2)\big)$$

- Hidden layer: two perceptrons (OR, NAND).
- Output: one perceptron (AND) on their outputs.
- The hidden layer changes the representation so that the problem becomes linearly separable.



Why not simply stack perceptrons and train them?

- The perceptron rule needs the *desired output of each unit*. For hidden units we do not know it.
- Training multilayer networks uses **gradient descent** on a loss function, with gradients computed by **backpropagation**.
- The step function is flat: a small change of the weights usually does not change the output at all, so it gives no hint in which direction to move. We replace it by a smooth **activation** (sigmoid, ReLU).



⇒ **Multilayer perceptron (MLP)** = layers of such units. Next lecture: Chollet & Watson, Ch. 2.

In Python

From scratch with NumPy

```
import numpy as np

def predict(X, w, b):
    return (X @ w + b >= 0).astype(int)          # Heaviside step

def fit_perceptron(X, d, eta=1.0, max_epochs=100):
    w, b = np.zeros(X.shape[1]), 0.0
    for epoch in range(max_epochs):
        mistakes = 0
        for x_j, d_j in zip(X, d):
            err = d_j - predict(x_j, w, b)            # -1, 0 or +1
            w += eta * err * x_j
            b += eta * err
            mistakes += abs(err)
        if mistakes == 0:                             # converged
            break
    return w, b, epoch + 1
```

With scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()
X = iris.data[:, 2:4]                       # petal length, petal width [cm]
y = (iris.target == 0).astype(int)          # 1 = setosa, 0 = other

clf = Perceptron(max_iter=1000, tol=1e-3, random_state=42)
clf.fit(X, y)
print(clf.coef_, clf.intercept_, clf.score(X, y))
```

- `Perceptron` is equivalent to `SGDClassifier(loss="perceptron", learning_rate="constant", eta0=1, penalty=None)`.
- By default the data are shuffled each epoch; `tol` stops training when the loss stops improving.

Summary

- Perceptron: $z = w_1 x_1 + \cdots + w_n x_n + b$, answer 1 if $z \geq 0$, else 0 — a linear classifier.
- Learning rule: only after a mistake, $w_i \leftarrow w_i + \eta\, e\, x_i$, $b \leftarrow b + \eta\, e$.
- Linearly separable data: it finds a separating line after finitely many updates.
- Not separable (XOR, overlapping Iris classes): it never settles.
- SVM: the separating line with the widest margin; soft margin (C) for overlapping classes.
- Several layers solve XOR; training them needs smooth activations and gradient descent $\Rightarrow$ MLP.

Lab: `Perceptron.ipynb`, `Iris_Perceptron.ipynb`

Optional: convergence theorem, precise statement

Notation: $\tilde{\mathrm{x}}_j = (1, x_{j1}, \ldots, x_{jn})$ (input with an extra 1 for the bias), $\tilde{\mathrm{w}} = (b, w_1, \ldots, w_n)$, so $z_j = \tilde{\mathrm{w}} \cdot \tilde{\mathrm{x}}_j$ (dot product); $\|\mathrm{v}\| = \sqrt{v_1^2 + \cdots + v_m^2}$ is the length of a vector; labels $t_j := 2d_j - 1 \in \{-1, +1\}$.

Theorem (Block 1962; Novikoff 1962)

Assume $\|\tilde{\mathrm{x}}_j\| \leq R$ for all j, and that the data are **linearly separable with margin** $\gamma > 0$: there is u with $\|\mathrm{u}\| = 1$ and $t_j\,(\mathrm{u} \cdot \tilde{\mathrm{x}}_j) \geq \gamma$ for all j. Then the perceptron algorithm started from $\tilde{\mathrm{w}} = 0$ makes at most

$$\left(\frac{R}{\gamma}\right)^2$$

updates, for any order in which the examples are presented.

Consequently it finds a separating hyperplane after finitely many epochs. The bound does not depend on the dimension or on the number of examples s.

Optional: proof sketch

On a mistake the update is $\tilde{w}_{k+1} = \tilde{w}_k + \eta\, t_j\, \tilde{x}_j$, and a mistake means $t_j\,(\tilde{w}_k \cdot \tilde{x}_j) \le 0$. After k updates:

1. **Alignment grows linearly:** $\tilde{w}_{k+1} \cdot u = \tilde{w}_k \cdot u + \eta\, t_j(\tilde{x}_j \cdot u) \ge \tilde{w}_k \cdot u + \eta\gamma \;\Rightarrow\; \tilde{w}_k \cdot u \ge k\eta\gamma$.
2. **Norm grows at most like $\sqrt{k}$:** $\|\tilde{w}_{k+1}\|^2 = \|\tilde{w}_k\|^2 + 2\eta \underbrace{t_j(\tilde{w}_k \cdot \tilde{x}_j)}_{\le 0} + \eta^2\|\tilde{x}_j\|^2 \le \|\tilde{w}_k\|^2 + \eta^2 R^2$

   $\Rightarrow\; \|\tilde{w}_k\|^2 \le k\eta^2 R^2$, i.e. $\|\tilde{w}_k\| \le \sqrt{k}\,\eta R$.
3. **Cauchy–Schwarz:** $k\eta\gamma \le \tilde{w}_k \cdot u \le \|\tilde{w}_k\| \le \sqrt{k}\,\eta R \;\Rightarrow\; k \le (R/\gamma)^2$. □

(For $d_j \in \{0, 1\}$ the rule from the previous slides gives exactly this update on a mistake, because then $d_j - \hat{y}_j = t_j$.)

References

- F. Rosenblatt (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review* 65(6), 386–408.
- H. D. Block (1962). The perceptron: a model for brain functioning. *Reviews of Modern Physics* 34(1), 123–135.
- A. B. J. Novikoff (1962). On convergence proofs on perceptrons. *Symposium on the Mathematical Theory of Automata*, 615–622.
- M. Minsky, S. Papert (1969). *Perceptrons*. MIT Press.
- C. Cortes, V. Vapnik (1995). Support-vector networks. *Machine Learning* 20(3), 273–297.
- F. Chollet, M. Watson (2025). *Deep Learning with Python*, 3rd ed. Manning.